

MedLift-3D

Visual Guide to the Codebase

Flow diagrams and a plain-English tour of every file

Who this is for

Anyone who needs to understand what this codebase does without reading the code — a supervisor, an examiner, a teammate picking it up, or you in six months. Eight flow diagrams first, then every file explained in ordinary language. No prior knowledge of medical imaging or diffusion models assumed.

In one line

The problem	A CT scan needs ~1000 X-rays and a real radiation dose. Checking whether a lung nodule is growing means scanning repeatedly.
The idea	Reconstruct a usable 3D volume from ~16 X-rays instead, using a model that has learned what chests look like, held in check by the actual measurements.
The catch	16 X-rays genuinely do not contain enough information. The model must guess. So every output ships a map of where it guessed.
The size	~5,800 lines. 15 library files, 9 scripts, 9 test suites.
The proof	77 tests, 35 seconds, no GPU or dataset needed. They must all pass before any training starts.

Contents

1 What this project does

Figure 1

2 The three stages

Figure 2

3 How the code is organised

Figure 3

4 Where the data comes from

Figure 4

5 The physics engine

Figure 5

6 How one reconstruction happens

Figure 6

7 What you actually run

Figure 7

8 Why there are so many tests

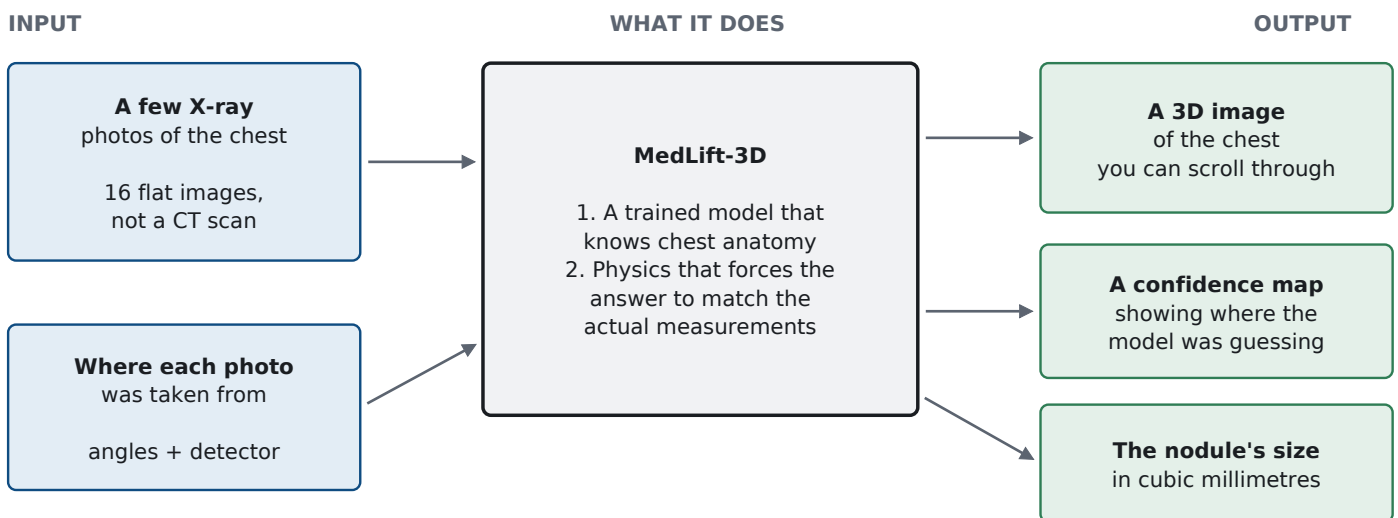
Figure 8

9 Every file, in plain English

9.1 foundations . 9.2 model . 9.3 extras . 9.4 plumbing . 9.5 scripts

1 What this project does

MedLift-3D takes a **handful of ordinary X-ray images** of someone's chest and builds a **3D picture** from them, then measures how big any lung nodule (a small lump) is. Normally that needs a CT scan, which is about a thousand X-ray images and a meaningful dose of radiation. The idea here is to get a usable answer from roughly sixteen.



The point: a CT scan needs ~1000 X-ray images and a big dose of radiation.
This tries to get a usable 3D result from about 16 of them.

Figure 1 — In one picture: a few flat X-rays go in, a 3D volume plus an honesty map plus a measurement come out.

Why a confidence map matters

Sixteen X-rays genuinely do not contain enough information to know everything about a chest. So the model has to guess to fill the gaps. The confidence map marks *where* it was guessing, so a doctor can tell a real finding from an invented one. Without it, the output would be an unfalsifiable picture.

2 The three stages

People get confused here, so it is worth being blunt: these are three **separate programs**, run at different times, on different data. They are not one button.

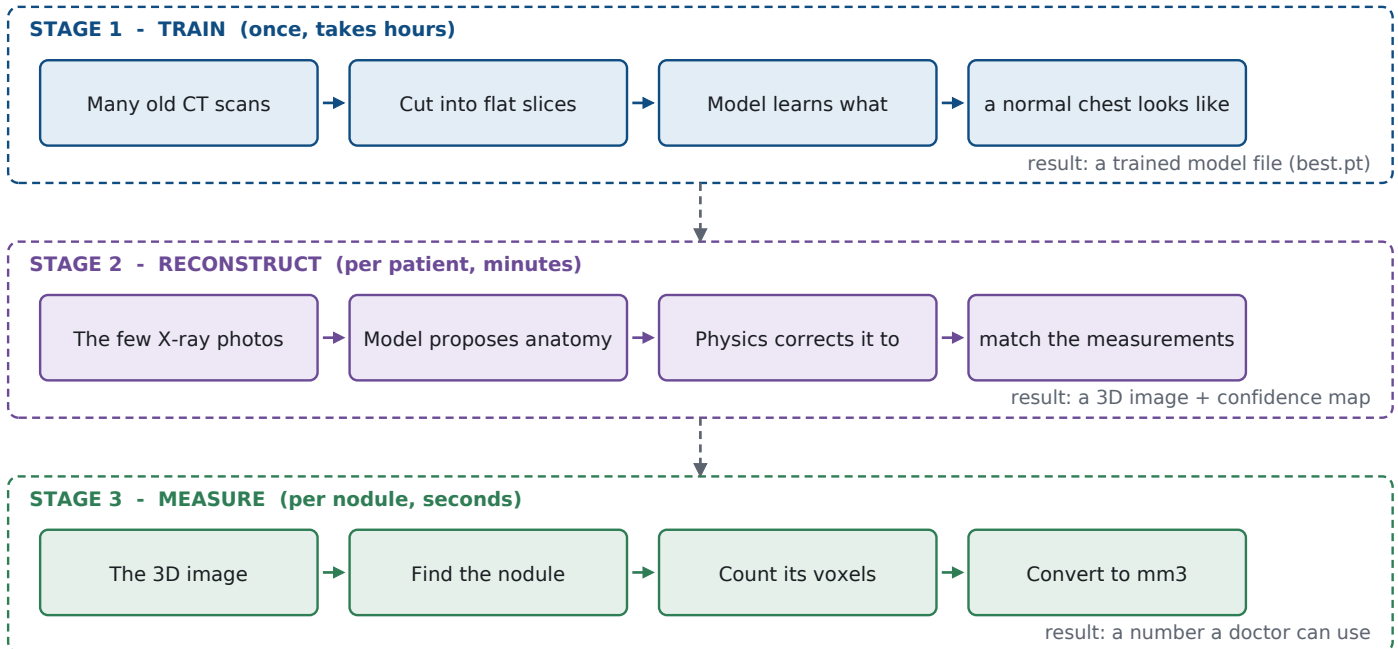


Figure 2 — Training happens once and is slow. Reconstruction happens per patient. Measuring happens per nodule.

Stage 1 is where all the time goes. You feed the model a few thousand slices of old CT scans until it has internalised what healthy and unhealthy chests look like. It never sees your patient. Stage 2 is where your patient appears, and it is fast because the hard learning already happened.

3 How the code is organised

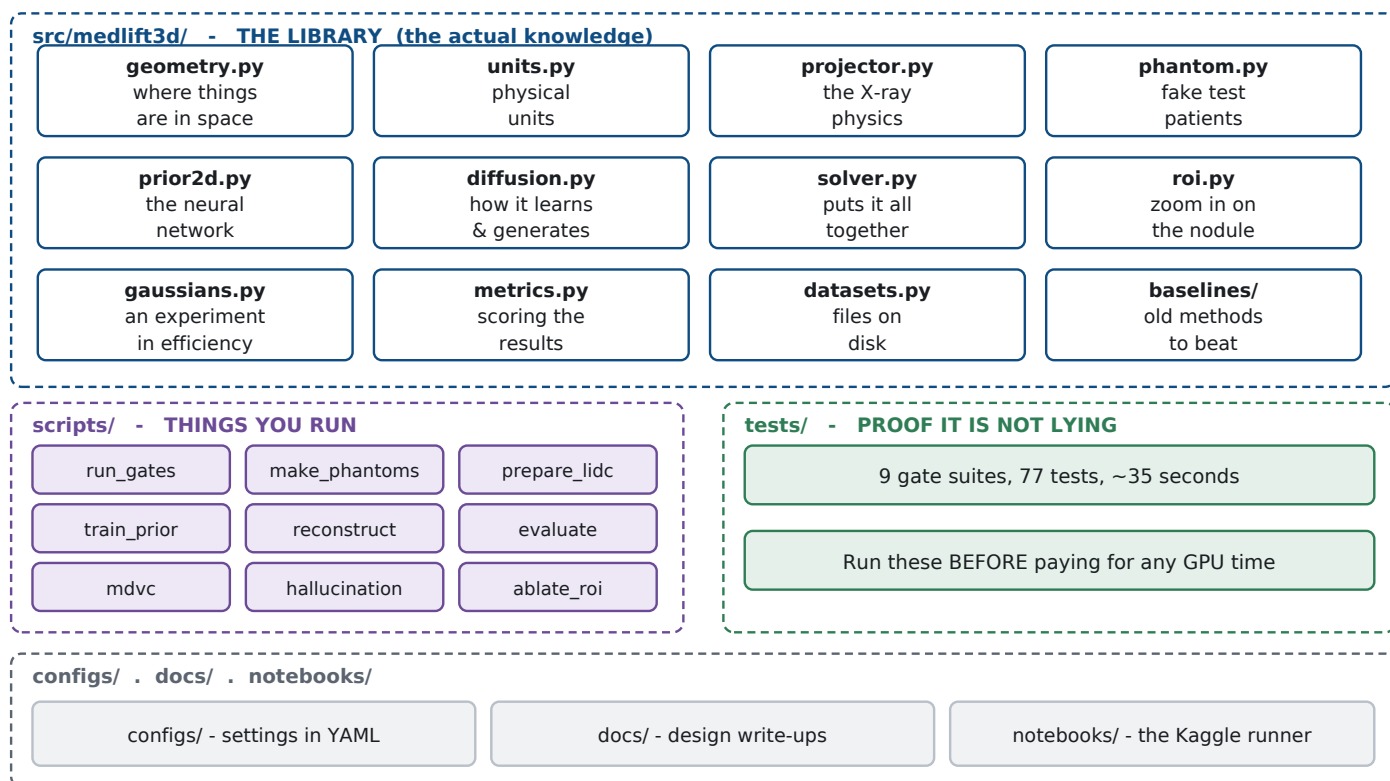
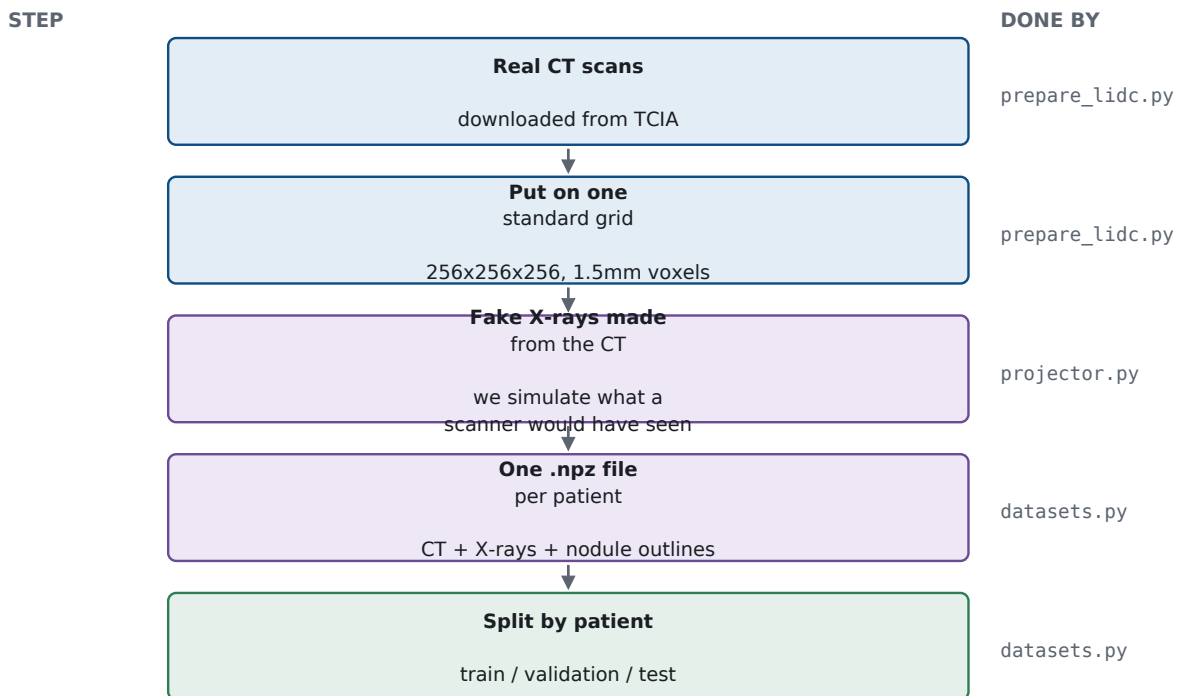


Figure 3 — Three layers. The library holds the knowledge, the scripts are what you actually type, the tests prove the library is not quietly lying to you.

The rule is that arrows only point one way: scripts use the library, the library never reaches back into the scripts. So you can read any library file on its own and understand it without knowing which experiment called it.

4 Where the data comes from



Why simulate the X-rays? Because no hospital has this scanner yet. We take a real CT, compute what a few-view scanner would have measured, and try to get the CT back.

Figure 4 — From a real CT scan on a public archive, to a tidy file per patient that everything downstream can read.

The bit that surprises people

We **simulate** the X-ray images from real CT scans. No hospital has a scanner that does this yet, so there is no real few-view data to train on. We take a CT, compute exactly what a few-view scanner would have measured, throw the CT away, and see if we can get it back. The CT is kept only to mark the homework.

5 The physics engine — the heart of it

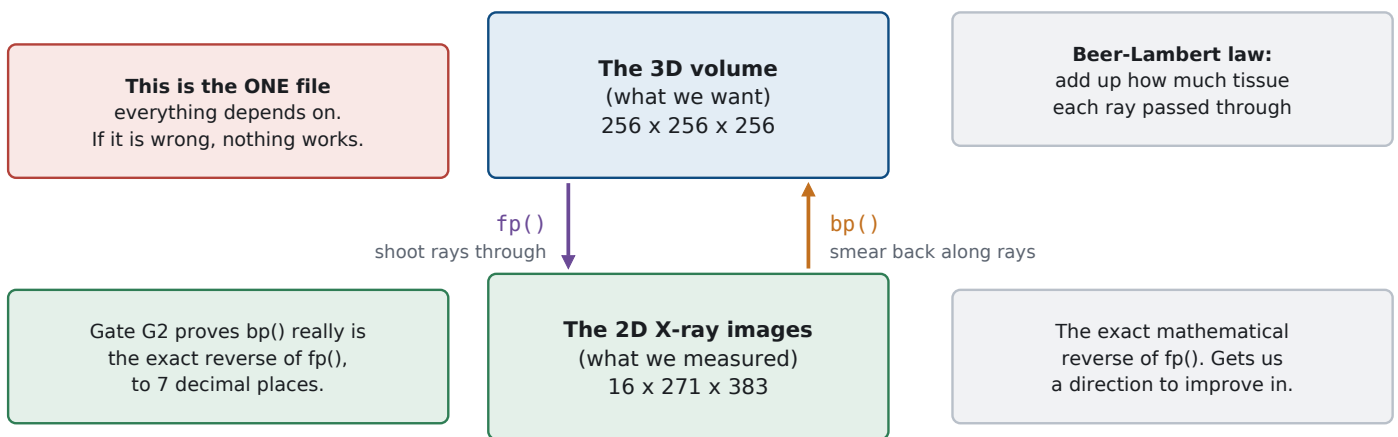
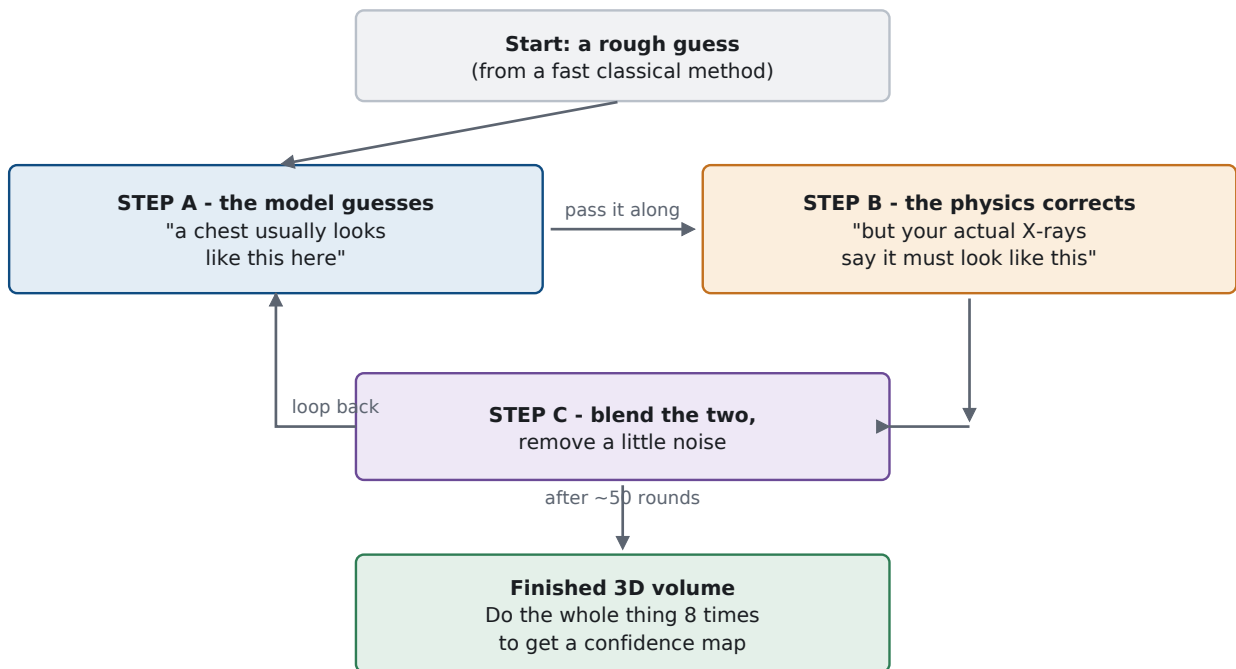


Figure 5 — Two operations, exact opposites of each other. Everything else in the project is built on this pair.

fp() answers: "if the chest looked like this, what would the X-ray machine have recorded?" **bp()** answers the reverse: "given this difference in the X-rays, which direction should I nudge the 3D volume?" The second one has to be the exact mathematical mirror of the first. If it is even slightly off, the whole thing appears to train normally and produces confident nonsense, which is precisely the failure the tests exist to catch.

6 How one reconstruction actually happens

Repeat about 50 times, each time a bit less noisy:

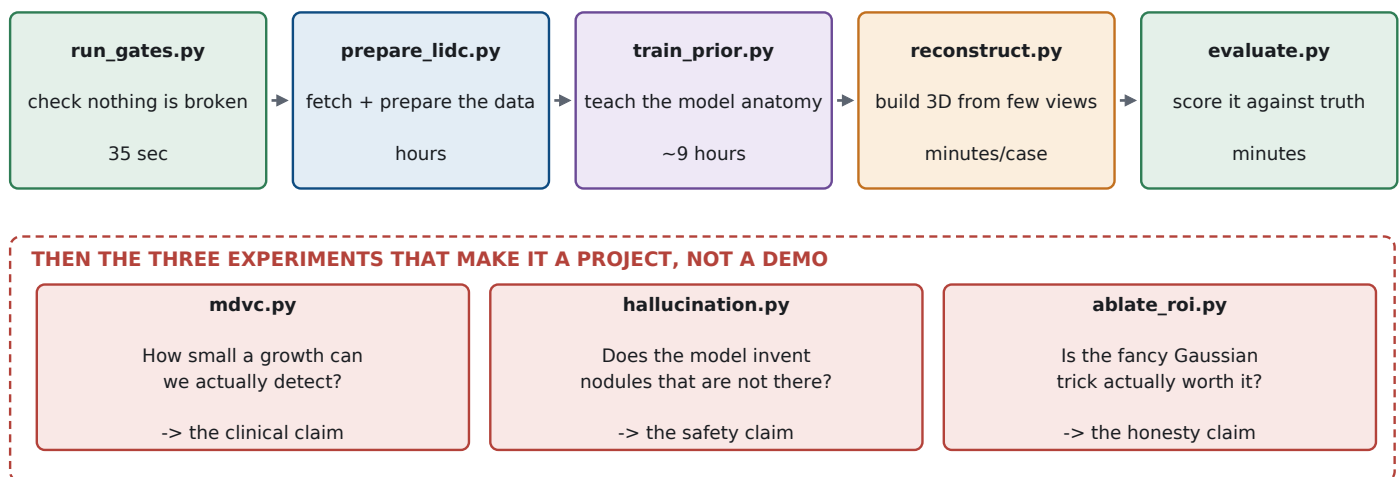


Why both halves? The model alone invents a convincing chest that is not YOUR chest. The physics alone cannot fill in what 16 photos never measured. Together they work.

Figure 6 — The loop that turns 16 flat images into a 3D volume. Two voices arguing, about fifty times.

Think of it as two experts in a room. One has seen ten thousand chests and says "there is almost certainly a rib here and a vessel there." The other has your actual X-rays and says "maybe, but whatever you draw has to be consistent with these measurements." Neither is right alone. The first one invents a plausible chest that is not yours; the second cannot fill in what was never measured. Alternating between them, fifty times, converges on something that is both anatomically sensible and true to your data.

7 What you actually run



Each writes a CSV plus a figure. These three are what the report is actually about.

Figure 7 — Left to right is the order you run them in. The bottom row is what turns this from a demo into a project.

8 Why there are so many tests

Four ways this project can fail SILENTLY - no crash, no error, just wrong answers

G1	geometry	The 3D image and the truth end up on different grids.	Every score you compute is meaningless.
G2	projector	The physics has no usable gradient.	The model trains on nothing and returns smooth mush.
G3	units	Density and Hounsfield units get mixed up.	Everything is scaled wrong; nodules vanish.
G4	metrics	The scoring itself is broken.	A perfect answer scores badly so you cannot tell good from bad.

This is why `scripts/run_gates.py` exists and why it runs first. 35 seconds, no GPU needed.

Figure 8 — Four failure modes that produce no error message. This is the reason the test suite is not optional.

Most bugs announce themselves — something crashes, a number is obviously wrong. The four above do not. They produce smooth loss curves, plausible pictures, and numbers you would happily put in a report. You would only find out at the very end, or never. Thirty-five seconds of tests before a nine-hour training run is an extremely good trade.

9 Every file, in plain English

The library is fifteen files. Here is what each one is for, what lives inside it, and why it exists at all.

9.1 The foundations

geometry.py

174 lines — knows where things are in the patient's body

A 3D image is just a big block of numbers. On its own it does not know how big a voxel is in millimetres, or which end is the head. This file attaches that information permanently, so a reconstruction and the truth can never accidentally be compared on different scales — which would silently make every score meaningless.

What's inside	What it does, plainly
Grid	The block of numbers plus its size, spacing and position in mm.
assert_matches	Shouts loudly if two images are on different grids.
CHEST, ROI	The two standard sizes used everywhere: a whole chest at 1.5 mm, and a zoomed box around a nodule at 1 mm.

units.py

74 lines — keeps the physics units straight

X-ray absorption can be written two ways: Hounsfield Units (what radiologists read) and attenuation coefficient (what the physics equations need). Mixing them up is a classic disaster. This file does all conversion in one place so nobody can get it wrong somewhere else.

What's inside	What it does, plainly
hu_to_mu / mu_to_hu	Convert between the doctor's units and the physics units.
HU_CLIP	The one brightness window used by everything, so the simulator and the scorer cannot drift apart.
apply_poisson	Adds realistic X-ray noise, correctly — noise has to be added before the logarithm, not after.

projector.py

397 lines — the X-ray physics; the most important file here

Simulates shooting X-rays through a 3D volume and recording what comes out the other side, and also does the exact reverse. Written in plain PyTorch so the maths stays differentiable, which is what lets the model learn through it. See Figure 5.

What's inside	What it does, plainly
fp()	Volume in, X-ray images out. "What would the scanner see?"
bp()	X-ray images in, volume out. The exact mirror of fp().
ParallelGeometry	Sparse CT: a few views spread over a half circle.
DTSGeometry	Tomosynthesis: views crammed into a narrow 30° arc. Much harder, because whole directions are never measured.

9.2 The model

prior2d.py

191 lines — the neural network that learns anatomy

A 48-million-parameter network that works on flat 2D slices rather than whole 3D blocks. That single choice is what makes this project runnable on one ordinary GPU instead of a research cluster — roughly 7 GB and a day, instead of 40 GB and a week.

What's inside	What it does, plainly
UNet2D	The network itself. Takes a noisy slice, predicts the noise.
cond_ch = 0	By default it is not given any hints. The measurements supply the patient-specific part later.

diffusion.py

228 lines — how the model learns and then generates

Diffusion works by teaching a network to undo noise. Add noise to a real CT slice, ask the network what noise you added, repeat a million times. To generate, start from pure noise and undo it step by step. This file handles both directions and, crucially, refuses to load a model that was trained with different settings than it is about to use.

What's inside	What it does, plainly
loss()	Used during training.
ddim_sample()	Used during generation.
load_state()	Refuses a mismatched checkpoint. This one guard prevents a bug that silently ruins an entire training run.

solver.py

219 lines — puts the model and the physics together

The reconstruction loop from Figure 6. Alternates between letting the model propose anatomy and forcing the result to agree with the measured X-rays, then runs the whole thing several times over to produce the confidence map.

What's inside	What it does, plainly
DiffusionSolver	Runs the loop.
reconstruct()	Does it 8 times; the average is the answer, the spread is the confidence map.
estimate_lipschitz	Works out how big a correction step is safe, instead of guessing a number that would need retuning per experiment.

9.3 The extras

roi.py

138 lines — zooms in on the nodule for a closer look

A whole chest at fine resolution will not fit in memory. But nodules are tiny and we already know roughly where they are. So: reconstruct the chest coarsely, then redo just a small box around the nodule at high resolution. Because the physics is linear, splitting it this way is mathematically exact, not an approximation.

gaussians.py

277 lines — a deliberately falsifiable experiment

Represents the zoomed-in region as a cloud of soft blobs instead of a grid of voxels, to see whether that is a better use of the same memory. The honest framing matters here: this is tested against a plain-voxel control at equal compute, and a result saying "the blobs did not help" is a perfectly good result to publish.

metrics.py

391 lines — scores the results, carefully

Scoring sounds easy and is not. Measure the whole image and the score is dominated by air, which every method gets right, so nothing tells methods apart. Threshold the whole chest looking for a nodule and you select the spine and the ribs instead. This file works only inside the lungs, and only in a small box around each nodule.

What's inside	What it does, plainly
global_metrics	Overall picture quality, measured inside the lungs only.
nodule_metrics	Two sets of numbers: one isolating reconstruction error, one showing raw agreement with the radiologists.
measure_nodule_volume	The clinical number, computed without peeking at the answer.
hallucinated_at	Did the model invent a nodule that was not there?

9.4 The plumbing

File	Lines	In one sentence
datasets.py	359	Reads and writes one file per patient, keeps the train/test split honest, and caches slices so training is not bottlenecked on unzipping the same volume over and over.
phantom.py	254	Builds fake but realistic chest data with nodules in known places, so the whole pipeline can be tested without downloading anything.
export.py	120	Writes the results in formats real medical software can open (NIfTI, DICOM).
utils.py	97	Small shared maths and the save/load of checkpoints.
logging_utils.py	55	Writes training progress to a CSV, because Kaggle has no internet for the usual tools and a killed session must leave its history behind.
baselines/	182	Four older, non-learning reconstruction methods. The new method has to beat these or it is not worth its complexity.

9.5 The scripts you type

Script	What it does	How long
run_gates.py	Runs all 77 tests and prints pass/fail. Run this first, always.	35 sec
prepare_lidc.py	Downloads real CT scans, puts them on the standard grid, simulates the X-rays.	hours
make_phantoms.py	Same output but from fake patients, so you can test with no download.	minutes
train_prior.py	Teaches the model anatomy. Resumes automatically if the session is killed.	~9 hours
reconstruct.py	Builds the 3D volume from the few X-rays, by any method.	minutes per case
evaluate.py	Scores everything and writes the comparison table.	minutes
mdvc.py	How small a change in nodule size can we actually detect?	hours
hallucination.py	Does the model invent nodules, or erase real ones?	hours
ablate_roi.py	Were the Gaussian blobs worth it? Honest answer either way.	~1 hour

If you read nothing else

projector.py is the physics and everything stands on it. **solver.py** is where the actual idea lives. **metrics.py** is what decides whether any of it worked. The other twelve files are support, and **scripts/run_gates.py** is what stops you wasting a day on a silent bug.